REST 实施详细信息

ONTAP Automation

NetApp
April 21, 2022

This PDF was generated from https://docs.netapp.com/zh-cn/ontap-automation/rest/rest_web_services_foundation.html on April 21, 2022. Always check docs.netapp.com for the latest.

目录

REST 实施详细信息 1
REST Web 服务基础 1
ONTAP REST API 版本控制 1
如何访问 ONTAP API 2
基本操作特征 3
控制 API 请求的输入变量 4
解释 API 响应 7
使用作业对象进行异步处理 9

REST 实施详细信息

REST Web 服务基础

表述性状态传输（ Representational State Transfer ， REST ）是一种用于创建分布式 Web 应用程序的模式。在设计 Web 服务 API 时，它会建立一组技术，用于公开基于服务器的资源并管理其状态。它使用主流协议和标准为管理 ONTAP 集群提供了灵活的基础。

> 虽然 REST 建立了一组通用的技术和最佳实践，但每个 API 的详细信息可能因开发期间所做的选择而异。在将 ONTAP REST API 用于实时部署之前，您应了解其设计特征。

资源和状态表示

资源是基于 Web 的系统的基本组件。创建 REST Web 服务应用程序时，早期设计任务包括：

- 识别系统或基于服务器的资源

  每个系统都使用和维护资源。资源可以是文件，业务事务，流程或管理实体。在设计基于 REST Web 服务的应用程序时，首先要完成的任务之一是识别资源。

- 资源状态和关联状态操作的定义

  资源始终处于数量有限的状态之一。必须明确定义状态以及用于影响状态更改的关联操作。

URI 端点

必须使用定义明确的寻址方案定义和提供每个 REST 资源。资源所在的端点和标识的端点使用统一资源标识符（ Uniform Resource Identifier ， URI ）。URI 提供了一个通用框架，用于为网络中的每个资源创建唯一名称。统一资源定位器（ Uniform Resource Locator ， URL ）是一种用于 Web 服务的 URI 类型，用于标识和访问资源。资源通常以类似于文件目录的分层结构公开。

HTTP 消息

超文本传输协议（ HTTP ）是 Web 服务客户端和服务器用来交换有关资源的请求和响应消息的协议。在设计 Web 服务应用程序时， HTTP 方法会映射到资源以及相应的状态管理操作。HTTP 为无状态。因此，要将一组相关请求和响应关联为一个事务的一部分，必须将追加信息包含在随请求和响应数据流一起提供的 HTTP 标头中。

JSON 格式化

虽然可以通过多种方式在 Web 服务客户端和服务器之间构建和传输信息，但最受欢迎的选项是 JavaScript 对象表示法（ JSON ）。JSON 是一种行业标准，用于以纯文本形式表示简单数据结构，并用于传输描述资源的状态信息。ONTAP REST API 使用 JSON 格式化每个 HTTP 请求和响应正文中包含的数据。

ONTAP REST API 版本控制

ONTAP 9 附带的 REST API 分配有一个版本号，该版本号与 ONTAP 版本号无关。您应了解您的 ONTAP 版本附带的 API 版本，以及此版本可能会对您的 API 使用产生何种影响。

所有版本的 ONTAP 9 都使用相同版本的 REST API 。

ONTAP 9.6 到 ONTAP 9.10.1

版本 1

相关链接

- "《 ONTAP 发行说明》"

如何访问 ONTAP API

您可以通过多种不同方式访问 ONTAP REST API 。

网络注意事项

您可以通过以下接口连接到 REST API：

- 集群管理 LIF
- 节点管理 LIF
- SVM 管理 LIF

您选择使用的 LIF 必须配置为支持 HTTPS 管理协议。此外，网络中的防火墙配置必须允许 HTTPS 流量。

您应始终使用集群管理 LIF 。这样可以在所有节点之间对 API 请求进行负载平衡，并避免出现脱机或连接问题的节点。

ONTAP API 联机文档页面

使用 Web 浏览器时， ONTAP API 联机文档页面提供了一个访问点。除了提供直接执行单个 API 调用的方法之外，此页面还包括 API 的详细问题描述，包括每个调用的输入参数和其他选项。API 调用按功能类别进行组织。请参见 "REST 资源摘要" 有关详细信息 …

用于访问最新版本 API 的文档页面的 URL 格式为：

```
https://<cluster_mgmt_ip_address>/docs/api
```

自定义软件和工具

您可以使用多种不同的编程语言和工具访问 ONTAP API 。常见选项包括 Python ， Java ， Curl 和 PowerShell 。使用 API 的程序，脚本或工具充当 REST Web 服务客户端。使用编程语言可以加深对 API 的了解，并提供自动化 ONTAP 管理的机会。

用于直接访问最新版本 API 的基本 URL 格式为：

```
https://<cluster_mgmt_ip_address>/api
```

要访问支持多个版本的特定 API 版本， URL 的格式为：

```
https://<cluster_mgmt_ip_address>/api/v1
```

基本操作特征

虽然 REST 建立了一组通用的技术和最佳实践，但每个 API 的详细信息可能因设计选择而异。

请求和响应 **API** 事务

每个 REST API 调用都作为 HTTP 请求执行给 ONTAP 系统，该系统会向客户端生成关联的响应。此请求 / 响应对被视为 API 事务。在使用 API 之前，您应熟悉可用于控制请求的输入变量以及响应输出的内容。

支持 **CRUD** 操作

通过 ONTAP REST API 提供的每个资源均可根据 CRUD 模型进行访问：

- 创建
- 读取
- 更新
- 删除

对于某些资源，仅支持部分操作。有关每个资源的详细信息，您应查看 ONTAP 集群上的 ONTAP API 文档页面。

对象标识符

创建每个资源实例或对象时，系统会为其分配一个唯一标识符。在大多数情况下，标识符为 128 位 UUID 。这些标识符在特定 ONTAP 集群中具有全局唯一性。发出创建新对象实例的 API 调用后，将在 HTTP 响应的位置标题中向调用方返回一个具有关联 ID 值的 URL 。在引用资源实例时，您可以提取此标识符并在后续调用中使用它。

> 对象标识符的内容和内部结构可以随时更改。仅当引用关联对象时，才应根据需要在适用的 API 调用上使用标识符。

对象实例和集合

根据资源路径和 HTTP 方法， API 调用可以应用于特定对象实例或对象集合。

同步和异步操作

ONTAP 可通过两种方式执行从客户端收到的 HTTP 请求。

同步处理

ONTAP 会立即执行此请求，如果成功，则会使用 HTTP 状态代码 200 或 201 进行响应。

使用 GET ， HEAD 和 OPTIONS 方法的每个请求始终会同步执行。此外，如果使用 POST ，修补和删除的请求预计在两秒内完成，则这些请求会同步运行。

异步处理

如果异步请求有效， ONTAP 将创建一个后台任务来处理此请求，并创建一个作业对象来锁定此任务。202 HTTP 状态将与作业对象一起返回给调用程序。要确定最终成功还是失败，您必须检索作业的状态。

如果使用 POST ，修补和删除方法的请求预计需要超过两秒才能完成，则这些请求会异步运行。

> `return_timeout` query 参数可用于异步 API 调用，并且可以将异步调用同步转换为完成。有关详细信息，请参见 如何使用作业对象进行异步处理 。

安全性

REST API 提供的安全性主要基于 ONTAP 提供的现有安全功能。API 使用以下安全性：

- 传输层安全性

  ONTAP LIF 与客户端之间通过网络发送的所有流量通常会根据 ONTAP 配置设置使用 TLS 进行加密。

- 客户端身份验证

  ONTAP System Manager 和网络易管理性 SDK 提供的相同身份验证选项也可用于 ONTAP REST API 。

- HTTP 身份验证

  在 HTTP 级别， API 事务使用基本身份验证。每个请求都会添加一个 HTTP 标头，其中用户名和密码包含在 base64 字符串中。

- ONTAP 授权

  ONTAP 实施基于角色的授权模式。访问 ONTAP REST API 或 API 文档页面时使用的帐户应具有适当的权限。

"使用 RBAC 实现安全性"

控制 API 请求的输入变量

您可以通过 HTTP 请求中设置的参数和变量来控制 API 调用的处理方式。

HTTP 方法

下表显示了 ONTAP REST API 支持的 HTTP 方法。

> 并非所有 HTTP 方法都可在每个 REST 端点使用。此外，修补程序和删除都可以用于集合。有关详细信息，请参见 _Object 引用和访问权限 _ 。

HTTP 方法	Description
获取	检索资源实例或集合上的对象属性。
发布	根据提供的输入创建新的资源实例。
patch	根据提供的输入更新现有资源实例。
删除	删除现有资源实例。
标题	有效发出 GET 请求，但仅返回 HTTP 标头。

HTTP 方法	Description
选项	确定特定端点支持哪些 HTTP 方法。

请求标题

您必须在 HTTP 请求中包含多个标头。

内容类型

如果请求正文包含 JSON ，则必须将此标头设置为 `application/json` 。

接受

此标头应设置为 `application/hal+json` 。如果将其设置为 `application/json` ，则除了检索下一批记录所需的链接之外，不会返回任何 HAL 链接。如果标头与这两个值不同，则响应中的 `content-type` 标头的默认值将为 `application/hal+json` 。

Authorization

必须使用用户名和密码设置基本身份验证，并将其编码为 base64 字符串。

请求正文

请求正文的内容因具体调用而异。HTTP 请求正文包含以下内容之一：

- 包含输入变量的 JSON 对象
- 空 JSON 对象

筛选对象

发出使用 GET 的 API 调用时，您可以根据任何属性限制或筛选返回的对象。例如，您可以指定一个要匹配的精确值：

` <field>=< 查询值 >`

除了精确匹配之外，还可以使用其他运算符返回一组值范围内的对象。ONTAP REST API 支持下表所示的筛选运算符。

运算符	Description
=	等于
<	小于
>	大于
⇐	小于或等于
>=	大于或等于
更新	或
!	不等于
*	贪婪的通配符

此外，您还可以在查询中使用 `null` 关键字或其否定 `！ null` 来根据是否设置了特定字段返回一组对象。

> 未设置的任何字段通常会从匹配查询中排除。

请求特定对象字段

默认情况下，使用 GET 发出 API 调用时，只会返回唯一标识一个或多个对象的属性以及 HAL 自链接。这组最小的字段可用作每个对象的密钥，并因对象类型而异。您可以通过以下方式使用 `fields` query 参数选择其他对象属性：

- 通用字段或标准字段

  指定 `fields=*` 以检索最常用的对象字段。这些字段通常保留在本地服务器内存中，或者几乎不需要处理即可访问。这些属性与使用带有 URL 路径密钥（ UUID ）的 GET 后为对象返回的属性相同。

- 所有字段

  指定 `fields=*` 可检索所有对象字段，包括需要额外服务器处理才能访问的字段。

- 自定义字段选择

  使用 `fields=<field_name>` 指定所需的确切字段。请求多个字段时，必须使用逗号分隔值，不能包含空格。

> 作为最佳实践，您应始终确定所需的特定字段。您只能在需要时检索一组通用字段或所有字段。哪些字段归类为通用字段，并使用 `fields=*` 返回，由 NetApp 根据内部性能分析确定。字段的分类可能会在未来版本中发生变化。

对输出集中的对象进行排序

资源收集中的记录将按对象定义的默认顺序返回。您可以使用 `order_by` 查询参数以及字段名称和排序方向更改顺序，如下所示：

`order_by-=<` **字段名称** `> asc=desc`

例如，您可以按降序对类型字段排序，然后按升序对 ID 排序：

`order_by-type desc ， id asc`

请注意以下事项：

- 如果指定了排序字段，但未提供方向，则这些值将按升序排序。
- 如果包含多个参数，则必须使用逗号分隔各个字段。

检索集合中的对象时分页

使用 GET 发出 API 调用以访问同一类型的一组对象时， ONTAP 会根据两个限制尝试返回尽可能多的对象。您可以对请求使用其他查询参数来控制其中的每个限制。特定 GET 请求达到的第一个限制将终止该请求，因此会限制返回的记录数。

如果请求在迭代所有对象之前结束，则响应将包含检索下一批记录所需的链接。

限制对象数量

默认情况下， ONTAP 最多为 GET 请求返回 10 ， 000 个对象。您可以使用 `max_records` query 参数更改此限制。例如：

```
max_records=20
```

根据相关时间限制以及系统中的对象总数，实际返回的对象数可以小于有效的最大数量。

限制检索对象所用的时间

默认情况下， ONTAP 会在获取请求允许的时间内返回尽可能多的对象。默认超时为 15 秒。您可以使用 `return_timeout` query 参数更改此限制。例如：

```
return_timeout=5
```

实际返回的对象数量可以小于有效的最大数量，具体取决于对对象数量以及系统中对象总数的相关限制。

缩小结果集的范围

如果需要，您可以将这两个参数与其他查询参数结合使用，以缩小结果集的范围。例如，以下内容最多返回在指定时间之后生成的 10 个 EMS 事件：

```
time⇒2018-04-04T15 ： 41 ： 29.140265Z&max_records=10
```

您可以通过问题描述发送多个请求来浏览各个对象。后续的每个 API 调用应根据最后一个结果集中的最新事件使用一个新的时间值。

大小属性

某些 API 调用以及某些查询参数使用的输入值为数字。您可以选择使用下表所示的后缀，而不是提供以字节为单位的整数。

后缀	**Description**
知识库	KB 千字节（ 1024 字节）或千字节
MB	MB 兆字节（ KB x 1024 字节）或兆字节
GB	GB 千兆字节（ MB x 1024 字节）或吉字节
TB	TB TB TB TB TB （ GB x 1024 字节）或 TB
PB	PB PB PB PB （ TB x 1024 字节）或对等字节

"对象引用和访问"

解释 API 响应

每个 API 请求都会生成对客户端的响应。您应检查响应以确定其是否成功，并根据需要检索其他数据。

HTTP 状态代码

下面介绍了 ONTAP REST API 使用的 HTTP 状态代码。

代码	原因短语	**Description**
200	确定	表示未创建新对象的调用成功。
201	已创建	已成功创建对象。响应中的位置标头包括对象的唯一标识符。
202	已接受	已启动后台作业以执行请求，但尚未完成。
400	请求错误	此请求输入无法识别或不适当。
401	未授权	用户身份验证失败。
403	已禁止	由于授权（ RBAC ）错误，访问被拒绝。
404	未找到	请求中引用的资源不存在。
405.	不允许使用此方法	此资源不支持请求中的 HTTP 方法。
409	冲突	尝试创建对象失败，因为必须先创建另一个对象或请求的对象已存在。
500	内部错误	服务器发生一般内部错误。

响应标头

ONTAP 生成的 HTTP 响应包含多个标头。

位置

创建对象时，位置标头包含新对象的完整 URL ，包括分配给该对象的唯一标识符。

内容类型

通常为 `application/hal+json` 。

响应正文

API 请求生成的响应正文内容因对象，处理类型以及请求的成功或失败而异。响应始终在 JSON 中呈现。

- 单个对象

  可以根据请求返回一个对象并显示一组字段。例如，您可以使用 GET 使用唯一标识符检索集群的选定属性。

- 多个对象

  可以从一个资源收集返回多个对象。在所有情况下，都会使用一致的格式，其中 `num_records` 表示包含对象实例数组的记录和记录的数量。例如，您可以检索特定集群中定义的节点。

- 作业对象

  如果异步处理 API 调用，则会返回作业对象，用于将后台任务固定。例如，用于更新集群配置的修补程序请求会异步处理并返回一个作业对象。

- 错误对象

  如果发生错误，则始终返回 Error 对象。例如，在尝试更改未为集群定义的字段时，您将收到错误消息。

- 空 JSON 对象

  在某些情况下，不会返回任何数据，并且响应正文包含一个空的 JSON 对象。

HAL 链接

ONTAP REST API 使用 HAL 作为机制来支持将 Hypermedia 作为应用程序状态引擎（HATEOAS）。返回用于标识特定资源的对象或属性时，还会包含一个 HAL 编码的链接，您可以轻松地找到并确定有关该资源的其他详细信息。

errors

如果发生错误，响应正文将返回一个错误对象。

格式。

错误对象的格式如下：

```
"error": {
"message": "<string>",
"code": <integer>[,
"target": "<string>"]
}
```

您可以使用代码值确定常规错误类型或类别，并使用消息确定特定错误。如果可用，目标字段将包含与错误关联的特定用户输入。

常见错误代码

下表介绍了常见错误代码。特定 API 调用可能包括其他错误代码。

代码		**Description**
1.	409	具有相同标识符的对象已存在。
2.	400	字段的值无效或缺失，或者提供了额外的字段。
3.	400	不支持此操作。
4.	405.	找不到具有指定标识符的对象。
6.	403	拒绝执行请求的权限。
8.	409	资源正在使用中。

使用作业对象进行异步处理

发出设计为异步运行的 API 请求后，始终会创建一个作业对象并将其返回给调用方。作业

将描述并固定处理请求的后台任务。根据 HTTP 状态代码，您必须检索作业状态以确定请求是否成功。

请参见 "API 参考" 确定要异步执行的 API 调用。

控制请求的处理方式

您可以使用 `return_timeout` query 参数控制异步 API 调用的处理方式。使用此参数可能会产生两种结果。

计时器将在请求完成之前过期

对于有效请求， ONTAP 将返回 202 HTTP 状态代码以及作业对象。您必须检索作业状态以确定请求是否成功完成。

请求在计时器过期之前完成

如果此请求有效并在该时间到期之前成功完成，则 ONTAP 将返回 200 HTTP 状态代码以及作业对象。由于请求已同步完成，如 200 所示，因此您无需检索作业状态。

> `return_timeout` 参数的默认值为零秒。因此，如果不包含参数，则始终会为有效请求返回 202 HTTP 状态代码。

正在查询与 **API** 请求关联的作业对象

HTTP 响应中返回的作业对象包含多个属性。您可以在后续 API 调用中查询 state 属性，以确定请求是否成功完成。作业对象始终处于以下状态之一：

非终端状态

- 已排队
- 正在运行
- 已暂停

终端状态

- success
- 失败

用于发出异步请求的常规操作步骤

您可以使用以下高级操作步骤完成异步 API 调用。此示例假设未使用 `return_timeout` 参数，或者此时间已到后台作业完成之前。

1. 问题描述一种设计为异步执行的 API 调用。
2. 接收 HTTP 响应 202 ，指示接受有效请求。
3. 从响应正文中提取作业对象的标识符。
4. 在一个定时环路中，在每个周期中执行以下操作：
   a. 获取作业的当前状态。
   b. 如果作业处于非终端状态，请重新执行环路。

5. 当作业达到终端状态（成功，失败）时停止。

"工作流 1： 更新集群联系人并检查作业状态"

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.